Analyse d'intégration PI-SPI (BCEAO) dans l'ERP SFD

Date : 2026-10-05 · Statut : analyse / proposition (aucune implémentation)

1. Qu'est-ce que PI-SPI

PI-SPI = Plateforme Interopérable du Système de Paiement Instantané de l'UEMOA. Infrastructure conçue, développée et opérée par la BCEAO (pas un produit tiers). Lancée officiellement le 30 septembre 2025 après une phase pilote en conditions réelles depuis le 5 juin 2025.

Caractéristique	Valeur
Périmètre	8 pays UEMOA, FCFA uniquement
Participants	Banques, établissements de monnaie électronique (EME), établissements de paiement, SFD agréés (supervisés par la Commission Bancaire)
Disponibilité	24/7/365
Délai d'exécution	< 10 s (max 20 s), crédit immédiat et irrévocable du bénéficiaire
Plafond	5 000 000 FCFA par transaction
Messagerie	ISO 20022 (pacs.008 / pacs.002 paiements, camt trésorerie, RACALIAS gestion alias)
Sécurité	mTLS + VPN, certificats, chiffrement bout en bout, supervision 24/7
Identifiants	Alias = numéro de téléphone ou adresse de paiement générée ; 1 alias par compte ; vérification du nom + pays du bénéficiaire avant confirmation ; annuaire central d'alias
Paiement marchand	QR code standardisé (logo octogonal), accepté par toutes les apps participantes, en présence physique
Services grand public	Transfert compte-à-compte, compte ↔ mobile money, paiement marchand QR
API Business	Homologuées depuis sept. 2026 (24 API sur 104 participants) : émission/réception de paiements, règlements de masse, automatisation, suivi temps réel. Sandbox : `developer.pispi.bceao.int`

Tarification (communiqué BCEAO du 2 oct. 2026, applicable au 2 nov. 2026)

Opération	Tarif
Envoi national ≤ 8 000 FCFA cumulés / jour / usager / participant	Gratuit, sans limite d'opérations (75 % des transactions ME de l'Union)
Envoi national > 8 000 FCFA	0 à 0,8 % HT, fixé par le participant
Réception particulier	Gratuite sans limite (ancienne règle : 30 réceptions/mois puis ≤ 1 %)
Réception marchand	Librement fixée par le participant
Transfrontalier	Mêmes conditions à partir du 1er juin 2027

Calendrier réglementaire

Échéance	Obligation
30 juin 2026	Date limite initiale de préparation technique/administrative pour tous les assujettis
30 sept. 2026	Ouverture effective des services pour banques, EME, établissements de paiement
2 nov. 2026	**Toute transaction de monnaie électronique interopérable passe obligatoirement par PI-SPI**
30 juin 2027	**Ouverture effective des services pour les SFD** supervisés par la Commission Bancaire (sanctions ensuite)

Adoption

Date	Participants connectés	dont SFD
31 juil. 2025 (pilote)	86	10 (Baobab, Cofina, UM-ACEP, UM-PAMECAS…)
2 avr. 2026	80 (59 banques, 9 EME, 11 SFD, 1 EP)	11
2 oct. 2026	175 autorisés	n.c.

Constat clé pour notre marché : au 2 avril 2026, seuls 11 SFD sur ~20 millions de clients du secteur étaient connectés ; zéro SFD connecté au Togo, Bénin, Niger, Guinée-Bissau. Freins identifiés : SI core banking non prêts, procédures LBC/FT et KYC non formalisées, gouvernance et reporting prudentiel non conformes, coût d'intégration. Conséquence de non-connexion : exclusion des partenariats mobile money, refinancement bancaire conditionné, marché de la diaspora fermé.

→ **Un ERP SFD « PI-SPI ready » est un argument commercial direct** : la connexion devient obligatoire pour les grands SFD d'ici juin 2027 et la majorité du marché n'a pas de solution.

Périmètre géographique : UEMOA uniquement

PI-SPI ne couvre que les 8 pays de l'UEMOA et ne traite que le franc CFA BCEAO (XOF). Une institution installée hors UEMOA (RDC, zone CEMAC, autre) ne peut ni s'y connecter ni l'utiliser : **la fonctionnalité doit lui être totalement invisible** (menus, écrans, options de mode de paiement, paramétrage, rapports) et refusée côté serveur.

Critère d'éligibilité retenu, déjà présent dans l'ERP : zone monétaire du **pays siège de l'institution principale** (`Institution.paysSiege` → `Pays.zoneMonetaire == UEMOA`, exposé par `PreferenceRestClient.findInstitutionPrincipale()` puis `findPaysByCode()` dans `sfd-integration-api`). La devise seule ne suffit pas comme critère (une institution UEMOA peut tenir des comptes en EUR/USD ; un pays CEMAC utilise XAF).

2. État actuel de l'ERP (scan du dépôt, 2026-10-05)

Constat : aucun rail monétaire externe réel. Ni client HTTP vers un opérateur mobile money, une banque ou la BCEAO, ni webhook entrant de paiement. « Mobile money » et « virement » n'existent que comme *modes de paiement comptables*.

Brique	Existant	Fichiers clés
Modes de paiement	`MOBILE_MONEY`, `VIREMENT`, `VIREMENT_EXTERNE` dans caisse, épargne, crédit, client, commercial, RH	`*/enums/ModePaiement.java`, `credit/.../ModeDecaisseme`

Brique	Existant	Fichiers clés
		`nt.java`, `epargne/.../ModeOperation.java`
Compte mobile money SFD	Entité float opérateur (ORANGE, MTN, MOOV, WAVE, AIRTEL), journal `MOB`, résolution par agence	`sfd-comptabilite-service/.../CompteMobileMoney.java`, `CompteMobileMoneyIntegrationController`
Contrepartie trésorerie	Publie `LignesComptablesEvent` pour MOBILE_MONEY / VIREMENT / CHEQUE	`ContrepartieTresorerieServiceImpl`, `TresorerieIntegrationController`
Opérations bancaires	`OperationBancaire` (VIREMENT_EMIS/RECU, PRELEVEMENT…), `CompteBancaire` (codeBanque, numeroCompte, iban)	`sfd-comptabilite-service`
Rapprochement bancaire	Import CSV/OFX, lettrage auto ; MT940 & CAMT.053 proposés par l'UI mais rejetés backend	`RapprochementBancaireServiceImpl.importerReleve`
Transferts	`sfd-transfert-service` interne (TRF_INT/AGC/NAT/UEM) ; pas de publication RabbitMQ effective	`TypeTransfertEnum`
Portail client	Virement **interne** compte→compte, retrait mobile money = **stub** (référence `RMM-…` + `SUCCESS`, aucun débit) ; `/rib` renvoie `RIB_NOT_FOUND`	`PortailRetraitMobileService` (24 lignes), `PortailVirementService`
Appli agent mobile	`modePaiement` = String libre, idempotence `clientRequestId`	`MobileDepotRequest`, `EpargneConnector`
Bus d'événements	`OperationEvent` CloudEvents (correlationId, modePaiement, `TresorerieInfo`, `BancaireInfo`), saga `SagaOperation`, dédup `OperationEventLog` (caisse, client seulement), **pas d'outbox** transactionnel	`sfd-integration-api/.../events/`
Identité	Client : `numeroClient`, `telephonePrincipal` ; comptes `DAV-YYYY-NNNNNN` ; **aucune clé RIB/IBAN générée**	`Client.java`, `SetupEpargneLoader`
Plan comptable	RCSFD classe 1 (114 Banques, 116 SFD, 117 autres IF) — **aucun compte EME / PI-SPI** ; extension SYSCOHADA 5521/5522	`accounting-norms/uemoa/rcs_sfd/plan-comptable.yml`
Banques	Référentiel `Banque` (codeBanque, codeBIC)	`sfd-commun-service/.../Banque.java`

Verdict : l'ERP a la *sémantique* (modes, événements, saga, contrepartie trésorerie, rapprochement) mais aucune *exécution* externe. PI-SPI serait le premier rail réel ; il remplace avantageusement N intégrations opérateur par opérateur (Orange, MTN, Moov, Wave…) par une seule connexion normalisée ISO 20022.

3. Modèles de connexion possibles

Modèle	Description	Pour qui	Effort ERP
A. Participant direct	Le SFD est participant PI-SPI : compte de règlement, certification BCEAO, mTLS/VPN, ISO 20022 complet, alias, QR, supervision 24/7	Grands SFD (Art. 44, supervisés Commission Bancaire) — obligation au 30 juin 2027	Élevé : connecteur ISO 20022 + exigences opérationnelles (HA, 24/7, réconciliation)

Modèle	Description	Pour qui	Effort ERP
B. Participant indirect / sponsorisé	Connexion via une banque participante qui porte le règlement ; le SFD expose ses comptes via l'API de la banque	SFD moyens non assujettis ou en attente d'agrément	Moyen : connecteur par banque sponsor (spécifique), moins de contraintes BCEAO
C. Client API Business	Le SFD agit comme *entreprise* cliente d'une banque homologuée (24 API Business au 17/09/2026) : émission/réception de paiements, règlements de masse, suivi	Petits SFD, démarrage rapide	Faible : REST JSON, pas de certification BCEAO ; mais pas d'alias/QR au nom du SFD, règlement sur compte bancaire du SFD

Recommandation : architecture ERP unique avec un **port « rail de paiement instantané »** et trois adaptateurs (A direct ISO 20022, B sponsor, C API Business). Démarrer par C (valeur immédiate, faible risque), bâtir A pour les SFD assujettis.

4. Architecture d'intégration proposée

4.1 Nouveau service `sfd-paiement-instantane-service` (ou module « rail »)

Responsabilités, isolées des modules métier :

1. **Gateway PI-SPI** : mTLS, certificats, VPN, signature ; mapping ISO 20022 ⇄ DTO interne (pacs.008 émission, pacs.002 statut, camt.054/053 notifications et relevés, RACALIAS alias).
2. **Annuaire & alias** : enregistrement/résolution alias (téléphone ou adresse de paiement) ↔ compte épargne `DAV-…` ; 1 alias par compte ; vérification nom bénéficiaire avant confirmation.
3. **Ordres sortants** : `OrdrePaiementInstantane` (statut INITIE → ENVOYE → ACCEPTE/REJETE → IRREVOCABLE), idempotence par `clientRequestId` + `endToEndId`, **outbox transactionnel** (absent aujourd'hui dans les modules financiers — prérequis pour un rail irrévocable).
4. **Crédits entrants** : réception pacs.008, contrôle alias/compte actif/plafond, crédit immédiat du compte épargne via épargne, accusé pacs.002 < 10 s.
5. **Règlement & réconciliation** : position de règlement BCEAO, rapprochement automatique des mouvements PI-SPI avec les lignes comptables (étend `RapprochementBancaire` : ajouter format CAMT.053 déjà proposé par l'UI).
6. **Tarification** : grille 0–0,8 % au-delà de 8 000 FCFA/jour cumulés, réception gratuite, marchand libre ; calcul cumul journalier par usager et participant.
7. **Supervision** : métriques temps de réponse (SLA 10 s), taux de rejet, file de retry, alertes.

4.1 bis Garde de zone (UEMOA)

Trois niveaux, pour qu'une institution RDC ou CEMAC ne voie ni n'appelle jamais PI-SPI :

Niveau	Mécanisme	Effet hors UEMOA
Activation du module	`ModuleRegistryService` (Angular) calcule aujourd'hui *licence* ∩ *santé du service* ; ajouter une 3e condition *éligibilité de zone* déclarée dans `nexora-module.registry.ts` (`zonesEligibles: ['UEMOA']`) et évaluée sur la zone de l'institution chargée au démarrage	Module absent du menu, routes bloquées par `ModuleGuard`, aucune sonde de santé lancée
Options dans les autres modules	Feature flag dérivé `pispiActif` (= module activé) lu par les écrans épargne (retrait PI-SPI), crédit (mode `MOBILE_MONEY`/`VIREMENT_EXTERNE` chaîné), paie (canal `PI_SPI`), commercial, portail client Angular et Flutter (retrait vers wallet, virement par alias, RIB/QR PI-SPI)	Options masquées ; les flux existants (espèces, virement interne, export

Niveau	Mécanisme	Effet hors UEMOA
		CSV paie) restent seuls proposés
Serveur (source de vérité)	`PiSpiZoneGuard` dans le service rail : refuse toute création de participant, ordre, alias, demande de paiement et tout webhook si l'institution n'est pas UEMOA (HTTP 403, code `PISPI_HORS_ZONE`) ; épargne/crédit/paie/portail refusent de même le mode `PI_SPI`. Résultat mis en cache, invalidé à la modification de l'institution ou du pays	Aucun appel possible même en forçant l'API ; le service rail peut ne pas être déployé du tout chez un client RDC

Conséquences : - Déploiement RDC/CEMAC : le service `sfd-paiement-instantane-service` n'est ni licencié ni démarré ; le monolithe le charge mais la garde le neutralise. - Plan comptable : les comptes et le journal `PIS` ne sont ajoutés qu'aux normes UEMOA (`accounting-norms/uemoa/*`), jamais aux normes `cemac` ni `rdc`. - Wizard de setup : l'étape de paramétrage PI-SPI n'apparaît que si le pays siège choisi est UEMOA. - Changement de pays siège d'une institution (cas rare) : désactive le module ; les ordres non terminés doivent être soldés avant. - Extension future : la même garde, paramétrée par zone, servira à un autre rail régional (ex. système de paiement instantané BEAC en zone CEMAC) sans toucher aux modules métier.

4.2 Flux métier via le bus existant

Réutilise `OperationEvent` (`modePaiement`, `TresorerieInfo`, `correlationId`) et la règle de comptabilisation dual-line.

Règle de routage : toute opération *client* (décaissement, remboursement, dépôt, retrait, cotisation) transite **obligatoirement par le compte épargne** du client ; PI-SPI ne touche jamais directement un dossier de crédit. Seules les opérations *institutionnelles* (paie, fournisseurs, trésorerie inter-agences) partent **directement du compte bancaire / de règlement** du SFD, sans compte épargne.

Flux	Chemin	Modules émetteurs des manches	Compte trésorerie
Dépôt externe → compte épargne	PI-SPI entrant → crédit `DAV-…`	Rail (manche trésorerie) + Épargne (manche compte client)	Compte de règlement PI-SPI « 11x » (RCSFD) / 52x (SYSCOHADA)
Retrait compte épargne → wallet/banque externe	Débit `DAV-…` → PI-SPI sortant	Épargne (manche compte + frais) + Rail (manche trésorerie)	idem
Décaissement crédit	1) Crédit → compte épargne (flux existant : crédit envoie manche dossier, épargne manche compte, aucune manche rail). 2) Le client retire ensuite vers son wallet = flux « retrait » ci-dessus, déclenché automatiquement si `ModeDecaissement = MOBILE_MONEY / VIREMENT_EXTERNE`	Étape 1 : Crédit + Épargne. Étape 2 : Épargne + Rail	Étape 1 : aucun. Étape 2 : règlement PI-SPI
Remboursement crédit	1) PI-SPI entrant → compte épargne = flux « dépôt » ci-dessus. 2) Prélèvement de l'échéance sur le compte épargne (flux existant `ModeRemboursement = PRL`, crédit + épargne) ; request-to-pay PI-SPI (`demande_paiement.write`) sert uniquement à provoquer l'étape 1	Étape 1 : Rail + Épargne. Étape 2 : Épargne + Crédit	Étape 1 : règlement PI-SPI. Étape 2 : aucun

Flux	Chemin	Modules émetteurs des manches	Compte trésorerie
Cotisation tontine / frais adhésion / règlement commercial client	PI-SPI entrant → compte épargne, puis prélèvement interne par le module métier	Rail + Épargne, puis Épargne + module	règlement PI-SPI puis aucun
Paie / RH	Compte bancaire du SFD → bénéficiaires (bulk `paiement_groupe.write`), **sans compte épargne** ; si l'employé est aussi client, l'alias peut cibler son `DAV-…` mais le flux reste institutionnel	Rail (manche trésorerie) + Paie (manche charge/tiers 42x)	Compte bancaire SFD (114) ou règlement PI-SPI
Fournisseurs (achats commercial)	Compte bancaire SFD → fournisseur, unitaire ou bulk	Rail + Commercial (manche tiers)	idem Paie
Trésorerie inter-agences / refinancement	Compte bancaire ↔ banque	Rail + Comptabilité	idem Paie

Conséquences : - `ModeDecaissement.MOBILE_MONEY / VIREMENT_EXTERNE` et `ModeRemboursement.MOB / VIR` ne désignent plus un paiement direct mais un **retrait/dépôt épargne chaîné** ; le dossier de crédit ne porte jamais d'identifiant PI-SPI (`endToEndId` vit sur `OperationEpargne`). - Le compte épargne garde la trace unique et le relevé client reste complet (argument réglementaire : traçabilité, LBC/FT, plafonds par client). - Les flux institutionnels (paie, fournisseurs) ont un compte trésorerie distinct : `TypeCompteBancaire.COURANT` du SFD, pas le compte de règlement client. Deux positions à rapprocher séparément. - La caisse n'intervient dans aucun flux (règle existante : la caisse ne publie que si espèces).

4.3 Modifications dans les modules existants

Module	Changement
Comptabilité	Compte(s) de règlement PI-SPI dans `plan-comptable.yml` RCSFD + SYSCOHADA ; journal `PIS` ; `TypeOperationBancaire.PAIEMENT_INSTANTANE_EMIS/RECU` ; import CAMT.053 ; `CodeOperateurMobile` → référentiel participants PI-SPI (BIC/identifiant participant)
Épargne	Endpoint crédit/débit atomique pour le rail (réutiliser dépôt/retrait `ModeOperation.VIREMENT` avec `clientRequestId`), table alias par compte, `/portail/comptes/{n}/rib` réel (clé RIB manquante)
Crédit	`DecaissementRequest` : alias/IBAN/BIC (collectés par l'UI mais absents du backend) transmis à l'épargne pour le retrait chaîné après décaissement sur compte épargne ; aucun appel direct au rail. Remboursement : réception sur compte épargne puis prélèvement `PRL` existant
Portail client (web + Flutter)	Remplacer le stub `PortailRetraitMobileService` par un ordre PI-SPI ; virement externe par alias ; QR de réception ; corriger le chemin Flutter `virements` vs `/virements/executer`
Agent mobile	Collecte terrain : `modePaiement` typé, option « le client paie par PI-SPI » (request-to-pay si BCEAO l'ouvre)
Transfert	`TRF_NAT` / `TRF_UEM` s'exécutent sur PI-SPI (plafond 5 M, transfrontalier à partir du 01/06/2027)
Intégration-api	`OperationEvent.TresorerieInfo` : ajouter `endToEndId`, `participantId`, `alias` ; outbox partagé
KYC / LBC-FT	Pré-requis réglementaire PI-SPI : KYC formalisé, seuils, reporting CENTIF ; vérifier ce que couvre le module Client (frein n°1 des SFD selon Financial Afrik)

5. Domaines d'utilisation pour un SFD

Domaine	Usage PI-SPI	Impact
Épargne	Dépôt depuis mobile money / banque vers le compte épargne par alias ; retrait vers wallet 24/7	Fin du stub retrait mobile ; collecte sans passage en agence
Crédit	Décaissement sur compte épargne puis retrait chaîné vers wallet/banque ; remboursement par dépôt sur compte épargne (alias ou request-to-pay) puis prélèvement d'échéance	Réduction cash, délai décaissement de jours à secondes, PAR amélioré, traçabilité complète sur le relevé épargne
Collecte terrain / tontine	Le client verse via PI-SPI sur le compte de collecte ; l'agent ne manipule plus d'espèces	Risque de fraude et de perte réduit
Transfert d'argent	Transferts nationaux puis UEMOA (2027) sans partenaire tiers	Revenus de transfert, diaspora
Paiement marchand	QR standardisé pour commerçants clients du SFD	Nouveau produit « compte marchand »
Paie / RH / fournisseurs	Règlement de masse des salaires et achats **directement depuis le compte bancaire du SFD**, sans compte épargne	Suppression des chèques et des virements manuels
Trésorerie	Virements inter-agences et vers banques de refinancement en temps réel	Pilotage de liquidité intraday
G2P / institutionnel	Réception d'aides sociales, paiement de taxes pour le compte des clients	Positionnement auprès des États

6. Bénéfices et risques

Bénéfices pour l'ERP (produit)

- **Conformité obligatoire** : SFD supervisés tenus d'ouvrir le service au 30 juin 2027 ; au 2 avril 2026, 11 SFD connectés seulement. Un ERP « PI-SPI ready » répond à une obligation que la majorité du marché n'a pas couverte.
- **Une intégration au lieu de N** : remplace les connecteurs par opérateur (Orange, MTN, Moov, Wave, T-Money) et les formats bancaires propriétaires par un standard unique ISO 20022, à coût BCEAO nul pour le participant.
- **Première exécution réelle** des modes `MOBILE_MONEY`/`VIREMENT_EXTERNE` déjà présents partout dans le code : valeur immédiate sans refonte des écrans.
- **Cohérence comptable** : s'insère dans la dual-line, la contrepartie trésorerie et le rapprochement existants.
- **Différenciation commerciale** : argument de vente face aux concurrents (cf. comparatifs `docs/comparatif-erp-sfd-*.pdf`).

Bénéfices pour le SFD (client)

- Coûts de transaction de 5–10 % (agrégateurs) à 0–0,8 %, réception gratuite.
- Disponibilité 24/7, irrévocabilité < 10 s, plafond 5 M FCFA.
- Accès aux partenariats mobile money, au refinancement bancaire et au marché de la diaspora (conditionnés à la connexion).
- Baisse du cash en agence : sécurité, coûts de caisse, clôture journalière simplifiée.

Risques et contraintes

Risque	Mitigation
Exigences opérationnelles participant direct (24/7, HA, supervision, compte de règlement, certification) hors périmètre d'un ERP seul	Proposer le modèle B/C en premier ; le modèle A avec hébergement managé (Kubernetes multi-site)
Irrévocabilité + absence d'outbox transactionnel dans les modules financiers → risque de double crédit / perte d'ordre	Outbox + idempotence `endToEndId` avant tout flux sortant
Pas de clé RIB/IBAN, pas d'annuaire alias	Livrer le référentiel alias et le RIB réel dès le lot 1
Plan comptable RCSFD sans compte EME/PI-SPI	Ajout de comptes + validation auditeur BCEAO
KYC/LBC-FT non formalisé chez beaucoup de SFD (frein principal)	Checklist conformité intégrée au module Client
Fraude (alias usurpé, ingénierie sociale)	Affichage nom bénéficiaire, OTP portail existant, plafonds par client, détection vélocité
Spécifications techniques BCEAO non publiques (sandbox API Business sur demande ; FAQ/alias inaccessibles depuis ce poste)	Demander l'accès sandbox `developer.pispi.bceao.int` et le kit participant avant le PLAN détaillé
Évolution tarifaire (grille revue au 02/10/2026)	Grille paramétrable, pas codée en dur
Fonction visible ou appelable par une institution hors UEMOA (RDC, CEMAC)	Garde de zone à 3 niveaux (§4.1 bis), tests e2e sur une base démo RDC qui vérifient l'absence totale du module et des options

7. Feuille de route proposée

Lot	Contenu	Prérequis	Durée indicative
0. Cadrage	Accès sandbox API Business + kit participant BCEAO ; choix modèle A/B/C par SFD cible ; comptes PCSFD	Contact BCEAO / banque sponsor	2–3 sem.
1. Fondations	Outbox transactionnel, `endToEndId`, référentiel participants, alias ↔ compte, RIB réel, comptes et journal PI-SPI, import CAMT.053	Lot 0	4 sem.
2. Rail API Business (modèle C)	Service rail + adaptateur REST banque homologuée ; crédit entrant → épargne ; retrait portail → wallet (remplace stub) ; décaissement crédit externe	Lot 1	6 sem.
3. Flux métier	Remboursement crédit, règlement commercial, paie bulk, transfert national ; rapprochement auto ; rapports PI-SPI	Lot 2	4 sem.
4. Participant direct (modèle A)	Adaptateur ISO 20022 (pacs/camt/RACALIAS), mTLS/VPN, QR marchand, supervision, qualification BCEAO	SFD assujetti + hébergement HA	8–12 sem. + certification
5. Transfrontalier	Activation TRF_UEM sur PI-SPI	01/06/2027	2 sem.

Dette existante à traiter dans le même périmètre : stub `PortailRetraitMobileService`, `RIB_NOT_FOUND`, chemin Flutter `virements`, IBAN/BIC ignorés par `DecaissementRequest`, MT940/CAMT.053 rejetés, `sfd-transfert-service` sans publication RabbitMQ.

Sources

- BCEAO, lancement officiel PI-SPI : https://www.bceao.int/fr/content/lancement-officiel-de-la-plateforme-interoperable-du-systeme-de-paiement-instantane-pi-spi
- BCEAO, connexion à PI-SPI (échéances) : https://www.bceao.int/fr/communique-presse/connexion-la-plateforme-interoperable-du-systeme-de-paiement-instantane-pi-spi-de
- BCEAO, communiqué du 1er août 2025 + liste pilote (10 SFD) : https://www.bceao.int/sites/default/files/2025-08/Communique+Annexe-Lancement_du_Syste%CC%80me_de_paiement_instantane%CC%81_31_Juillet_2025.pdf
- Sandbox API Business : https://developer.pispi.bceao.int/guides/vue-ensemble
- Nouvelles dispositions oct. 2026 (obligation 2 nov., tarifs) : https://fr.allafrica.com/stories/202610040114.html ; https://www.financialafrik.com/2026/10/04/la-bceao-rend-pi-spi-obligatoire-revoit-ses-tarifs-et-pose-enfin-la-question-de-nouveaux-services-financiers-adosses-a-la-plateforme/
- API Business homologuées : https://notreafrik.com/pi-spi-bceao-api-business-homologuees/
- Pourquoi les SFD restent à quai : https://www.financialafrik.com/2026/04/15/pi-spi-uemoa-pourquoi-les-sfd-restent-a-quai/
- Participants avril 2026 : https://financesao.com/bceao-pi-spi-participants-autorises-avril-2026/
- Fiche fonctionnelle (alias, QR, plafonds, sécurité) : https://mondedesbanques.com/culture-generale/pi-spi-tout-ce-quil-faut-savoir/
- Architecture technique (ISO 20022, mTLS, phases) : https://gracelab.io/pi-spi
- Enjeux : https://lemarche.finance/pi-spi-de-la-bceao-enjeux-et-potentiels-impacts-dun-projet-ambitieux/

8. Démarches : sandbox et kit participant (vérifié le 2026-10-05 sur le portail BCEAO)

8.1 Sandbox API Business (modèle C) — libre-service, gratuit

Portail : https://developer.pispi.bceao.int (inscription : https://developer.pispi.bceao.int/sign-in → « Créer un compte », email professionnel, accès immédiat).

Parcours officiel affiché par le portail : 1. Créer le compte (email pro). 2. Se connecter à un **participant simulateur** → génération automatique `clientId`, `clientSecret`, clé API, compte **PICERT**. 3. Se connecter à PICERT (autorité de certification BCEAO) → générer le certificat client mTLS (validité 365 j). 4. Choisir un cas d'usage (point de vente QR, e-commerce, salaires groupés, facturation / demandes de paiement) et suivre le tutoriel. 5. Intégrer et tester : playground, simulateurs clients, SDKs, webhooks signés avec retry. 6. Production : **contacter un participant homologué** de la liste officielle (la BCEAO ne distribue pas l'API aux entreprises ; c'est la banque/EME homologuée qui fournit credentials + certificats de prod).

Faits techniques (guide « Sécurité ») : OAuth 2.0 `client_credentials` sur `https://api.pispi.bceao.int/oauth/token`, token lié au certificat mTLS (certificate-bound), en-tête `X-API-Key` ; scopes `compte.*`, `alias.*`, `paiement.*`, `paiement_groupe.*`, `demande_paiement*.*` (request-to-pay unitaire et en masse), `retour_fonds.write`, `demande_annulation*.write`, `webhook.*` ; rate limit sandbox 100 req/min – 10 000/jour, prod 1 000/min – 100 000/jour (négociable) ; spécification OpenAPI homologuée. Guides disponibles : vue d'ensemble, sécurité, gestion des certificats, permissions, alias, QR code, paiements, demandes de paiement, retours & annulations, webhooks, gestion des comptes, simulateurs.

Contact sandbox : `pisfn-sandbox@bceao.int` (lien « Contact » du portail) ; support détaillé via « Centre d'aide → Nous contacter » après connexion.

8.2 Kit participant direct (modèle A) — pas en libre-service

Aucun kit public : la participation directe est une démarche institutionnelle du SFD (pas de l'éditeur) auprès de la BCEAO.

1. **Le SFD** (agréé, supervisé Commission Bancaire) adresse une demande d'adhésion à la **Direction Nationale BCEAO de son pays** (service systèmes et moyens de paiement), qui transmet au Siège (Dakar, Département des Systèmes et Moyens de Paiement). Courrier officiel signé DG, mentionnant l'agrément et le SI utilisé.
2. La BCEAO remet le dossier participant : règles de fonctionnement PI-SPI, spécifications techniques ISO 20022 (pacs/camt/RACALIAS), exigences sécurité (mTLS, VPN, PICERT), convention de participation, compte de règlement, plan de tests et de qualification.
3. Phases : intégration & connexion (sandbox participant) → tests & validation fonctionnelle → qualification BCEAO → mise en production + inscription sur la liste officielle des participants autorisés (https://www.bceao.int/fr/communique-presse/liste-des-participants-autorises-ouvrir-les-services-de-pi-spi-au-public).
4. Pré-requis non techniques exigés dans les faits : procédures LBC/FT et KYC formalisées, responsable conformité, reporting CENTIF, gouvernance et ratios prudentiels conformes (frein n°1 des SFD selon Financial Afrik, avr. 2026).

Pour l'éditeur ERP : s'appuyer sur un SFD client pilote déjà engagé (ou un SFD de la liste pilote : Baobab, Cofina, UM-ACEP, UM-PAMECAS) pour obtenir le kit via sa demande, ou solliciter directement la Direction Nationale au titre de **fournisseur de SI** d'un SFD candidat. Alternative sans kit : modèle B (banque sponsor) ou C (API Business) décrits en §3.

8.3 Actions immédiates

#	Action	Qui	Résultat attendu
1	Créer le compte sandbox (email pro) sur developer.pispi.bceao.int	Éditeur	Credentials simulateur + certificat PICERT + OpenAPI
2	Exporter la spec OpenAPI et les guides (paiements, RTP, webhooks, alias) dans `sfd-docs/refs/pispi/`	Éditeur	Base du PLAN lot 1–2
3	Écrire à `pisfn-sandbox@bceao.int` : demande d'accès participant/kit au titre de fournisseur SI pour SFD	Éditeur	Orientation vers la Direction Nationale compétente
4	Courrier d'adhésion du SFD pilote à la Direction Nationale BCEAO	SFD client	Dossier participant, convention, calendrier de qualification
5	Identifier une banque homologuée API Business (Ecobank présent dans les 8 pays) pour le modèle C en production	Éditeur + SFD	Accès production sans certification BCEAO directe

Annexe — Plan d'implémentation (lots 0-3)

Plan d'implémentation — Intégration PI-SPI (API Business, lots 0-3)

Source : `sfd-docs/ANALYSE_INTEGRATION_PI-SPI_2026-10.md` (à commiter en premier, voir Lot 0).

Contexte

PI-SPI (BCEAO) devient obligatoire : monnaie électronique interopérable dès le 2 nov. 2026, SFD supervisés au 30 juin 2027. L'ERP n'a aucun rail monétaire externe : `MOBILE_MONEY` / `VIREMENT_EXTERNE` sont des modes comptables, le retrait mobile du portail est un stub, pas de clé RIB, pas d'alias, pas d'outbox dans les modules financiers. Objectif : un nouveau microservice `sfd-paiement-instantane-service` branché sur l'**API Business** (OAuth2 client_credentials + mTLS + X-API-Key, webhooks signés) d'un participant homologué (modèle C), piloté par les flux métier existants via le bus RabbitMQ et la comptabilisation dual-line.

Décisions prises avec l'utilisateur : périmètre lots 0-3 (pas de participant direct ISO 20022) ; nouveau microservice ; canaux Angular back-office + portail client Angular + Flutter portail client (agent mobile exclu).

Périmètre (invariant) : PI-SPI = UEMOA uniquement. Institution dont le pays siège n'est pas en zone `UEMOA` (RDC, CEMAC…) → module, écrans, options et endpoints invisibles et refusés. Critère : `Institution.paysSiege` → `Pays.zoneMonetaire` via `PreferenceRestClient.findInstitutionPrincipale()` + `findPaysByCode()`.

Règle de routage (invariant) : toute opération *client* (dépôt, retrait, décaissement, remboursement, cotisation) passe par le compte épargne `DAV-…`. Seules les opérations *institutionnelles* (paie, fournisseurs, trésorerie) partent du compte bancaire du SFD. Le dossier de crédit ne porte jamais d'identifiant PI-SPI.

Découpage : librairie commons `pispi-api` + intégration EDIP

PI-SPI est réutilisable hors EDIP (e-commerce, paie d'entreprise, facturation, autres projets kodzotech). Le connecteur technique va donc dans `~/Projects/Java/commons/pispi-api` (`com.kodzotech:pispi-api`, package `com.kodzotech.pispi`), sur le modèle de `notification-api` / `paiement-api` (Boot 3.5.9, Java 21, Jenkinsfile, auto-config). EDIP la consomme comme les autres libs.

Pourquoi une lib dédiée et pas un 5e provider de `paiement-api` : `paiement-api` est orienté *encaissement par redirection* (interface `PaiementClient` = `enregistrerPaiement` + `getPaiementStatus`, URL de paiement). PI-SPI couvre en plus les virements sortants, la paie en masse, les alias, les demandes de paiement, les retours/annulations, les webhooks signés et le mTLS par participant : l'interface `PaiementClient` serait trop étroite. En option (lot 2), `paiement-api` ajoute un provider `PISPI` (`PaiementClientEnum.PISPI`) qui délègue à `pispi-api` pour le cas simple « encaisser par QR / demande de paiement », sans dupliquer le client.

Dans `pispi-api` (générique, aucune notion SFD)	Dans EDIP `sfd-paiement-instantane-service` (métier)
Client API Business : OAuth2 client_credentials + cache token, mTLS (keystore PKCS12 par participant), `X-API-Key`, retry idempotent par `endToEndId`	Routage métier : compte épargne pour les flux clients, compte bancaire SFD pour paie/fournisseurs
DTO générés depuis l'OpenAPI BCEAO (openapi-generator au build)	Appels épargne / crédit / paie / commercial, `OperationEvent`, `LignesComptablesEvent`, saga
Façade `PiSpiClient` : paiements unitaires et groupés, statut, alias (créer/vérifier/supprimer), demandes de paiement (unitaire, masse, réponse), retours de fonds, annulations, comptes/transactions, webhooks (enregistrement)	Garde de zone UEMOA (Institution/Pays de Préférence), rôles `ROLE_PISPI_*`, écrans Angular, portail
`PiSpiWebhookVerifier` (signature, horodatage, anti-rejeu) + `PiSpiWebhookDispatcher` (événements Spring typés : `PaiementRecuEvent`, `PaiementStatutEvent`, `DemandePaiementReponseEvent`, `RetourRecuEvent`)	Réaction aux événements : crédit épargne, comptabilisation, retour si alias inconnu
`TarificationPiSpi` paramétrable (seuil gratuit/jour, taux, réception) + `PlafondPiSpi` (5 M/transaction)	Cumul journalier par client EDIP, frais publiés en compta
Générateur/lecteur QR PI-SPI	Affichage QR portail / RIB
Persistance optionnelle (`pispi.persistence.enabled`) : `PiSpiTransaction`, `PiSpiWebhookEvent` (dédup), outbox d'envoi ; tables préfixées `pispi_`	`OrdrePaiement` métier (lien compte épargne, dossier, bulletin, correlationId), `PaiementGroupe` paie
`PiSpiParticipantResolver` (interface) : l'appelant fournit config + secrets par participant ; impl par défaut depuis `application.yml`	Impl EDIP : `ParticipantConfig` en base, secrets chiffrés `SecretCipher`
`PiSpiEligibilityProvider` (interface, défaut = toujours éligible) appelée avant tout appel sortant	Impl EDIP : zone UEMOA de l'institution (cf. 1.4)
`PiSpiSimulateur` (profil test/démo) + `PiSpiTestSupport` (WireMock stubs) réutilisables	Tests d'intégration EDIP sur le simulateur

Règles de la lib : aucune dépendance `com.synapsesit`, aucun RabbitMQ (événements Spring seulement), pas de `@ComponentScan` large (auto-config `META-INF/spring/…AutoConfiguration.imports`), propriétés `pispi.*`, versions sémantiques `1.0.0-SNAPSHOT` → `1.0.0`. Entrée dans `commons/INDEX.md`.

Conventions réutilisées (ne pas réinventer)

- Scaffold : copier `sfd-transfert-service` (pom parent Boot 3.5.9, Java 21, `com.synapsesit.sfd.<module>.{controller,service.impl,repository,dto,entity,mapper,integration}`, configs dans `com.synapsesit.sfd.config`, `Setup<Module>Loader` pour les rôles).
- Port **4615** app / contexte `/api/paiement-instantane` ; ports hôte Jenkins dev **4636** / prod **4637** (4615 collisionne avec compta prod).
- Clients inter-services : `sfd-integration-api` (`ComptabiliteRestClient`, `EpargneRestClient`, `ClientRestClient`), pattern `<X>Config` + `<X>Properties` + `RestClient` avec `ServiceKeyPropagationInterceptor → AgenceHeaderInterceptor → ServiceTokenFallbackInterceptor`.
- Événements : `OperationEvent` (exchange `sfd.operations`, key `operation.<type>`), `LignesComptablesEvent` (exchange `sfd.comptabilite`, key `comptabilite.lignes.<module>`), `SagaOperation` + `OperationPublisherService` + `LignesComptablesPublisherService` (copier depuis épargne).

- Outbox : copier `WorkflowOutbox` + relay AFTER_COMMIT + `OutboxRetryJob` de `sfd-workflow-service` (`WorkflowOutcomeService`).
- Webhook signé : pattern `WebhookService.hmacHex(secret, ts, body)` + `WebhookSecurityConfig` (`@Order(HIGHEST_PRECEDENCE)`, `permitAll`) + `SecretCipher` AES-GCM de workflow.
- Tests : `@ExtendWith(MockitoExtension)` services, `@WebMvcTest(excludeAutoConfiguration=Security…)` contrôleurs, `@DataJpaTest` H2 `MODE=MSSQLServer`, `AbstractMultiDbSchemaTest` tagué `testcontainers`.
- Angular : `features/<module>/{module,routing,store/<entité>/{action,effect,model,reducer,selector,service},view/…}` sur le modèle `features/comptabilite/view/tresorerie/compte-mobile-money-liste` ; enregistrement dans `feature-routes.registry.ts`, `nexora-module.registry.ts`, `deployment.config.ts`, `environment*.ts`, `proxy*.conf.json`, `nginx-prod.conf`, `menu-module-extensions.ts`, `assets/i18n/{fr,en,ln}.json`, `e2e/helpers/service-map.ts` + `module-domains.ts`.

Lot 0 — Cadrage (sans code applicatif, 1 semaine)

1. Commit `sfd-docs/ANALYSE_INTEGRATION_PI-SPI_2026-10.md` (message `docs(pi-spi): routage des flux — décaissement/remboursement via compte épargne, paie directe compte bancaire`).
2. Compte sandbox `developer.pispi.bceao.int` (créé par l'utilisateur) → récupérer et versionner dans `sfd-docs/refs/pispi/` : spec OpenAPI API Business, guides paiements / demandes de paiement / alias / webhooks / retours-annulations / gestion des comptes, format QR.
3. Figer les contrats à partir de l'OpenAPI : noms exacts des endpoints, schéma webhook (signature, en-têtes, types d'événements), codes statut d'un paiement, `endToEndId`/référence, format alias.
4. Décision comptable à valider avec l'auditeur : compte de règlement PI-SPI clients (RCSFD extension sous 11x, ex. `1181 Compte de règlement PI-SPI` ; SYSCOHADA `5213`) + journal `PIS` ; comptes de frais PI-SPI (charge 6xx) et produits frais marchand (7xx).

Sortie : `sfd-docs/refs/pispi/CONTRAT_API_BUSINESS.md` (tableau endpoint ↔ cas d'usage ERP) qui alimente les lots 1-3.

Lot 1 — Fondations transverses (3-4 semaines)

1.1 sfd-integration-api (`com.synapsesit.sfd.integration`)

- `events/OperationEvent.TresorerieInfo` : ajouter `endToEndId`, `participantId`, `aliasBeneficiaire`, `canal` (`PI_SPI`).
- Nouveau type d'opération dans les enums consommés : `DEPOT_PI_SPI`, `RETRAIT_PI_SPI` (épargne), `PAIEMENT_INSTITUTIONNEL_PI_SPI` (paie/commercial/compta).
- Nouveau client `paiementinstantane/{PaiementInstantaneConfig, Properties, RestClient, Service}` (`app.paiement-instantane.base-url`, défaut `http://localhost:4615/api/paiement-instantane`), ajouté au fichier `META-INF/spring/…AutoConfiguration.imports`. Méthodes : `initierPaiement(OrdrePaiementRequest)`, `getStatut(endToEndId)`, `resoudreAlias(alias)`, `creerAlias(numeroCompte, alias)`, `initierPaiementGroupe(List<…>)`.
- Outbox partagé : `common/outbox/{OutboxEvent (entity abstraite ou table par service), OutboxRelay, OutboxRetryJob}` extrait de `WorkflowOutcomeService` ; auto-config conditionnelle

`app.outbox.enabled`. Version `1.0.2-SNAPSHOT`, bump dans les poms consommateurs (épargne, rail, paie).

1.2 Épargne (`sfd-epargne-service`)

- Idempotence retrait : ajouter `clientRequestId` à `RetraitRequestDto` + `EpargneRetraitRequest` ; `effectuerRetrait` applique `repository.findByClientRequestId` comme `effectuerDepotTerrain`.
- `ModeOperation.PI_SPI` (ou réutiliser `VIREMENT` + `canal`) — recommandation : nouveau mode `PI_SPI` pour schéma comptable dédié ; `exigerTresorerie` accepte `compteReglementPiSpiId`.
- Table alias : `compte/entity/AliasCompte {numeroCompte, typeAlias(TELEPHONE|ADRESSE_PAIEMENT), valeur, participantId, statut, dateCreation}` + repository + `AliasCompteService` (1 alias par compte, unicité valeur) ; endpoint `POST/DELETE /portail/comptes/{n}/alias` (client) et `/comptes/{n}/alias` (back-office). Rôles `ROLE_EPARGNE_ALIAS_*` dans `SetupEpargneLoader`.
- RIB réel : `GET /portail/comptes/{n}/rib` dans `PortailCompteEpargneController` (manquant aujourd'hui) → `PortailRibDto` ; génération clé RIB (modulo 97) dans `CompteEpargneService.genererRib()` à partir de `Banque.codeBanque/codeAgence/numeroCompte` ; corriger le chemin `/virements/frais` appelé par le portail (préfixe `/portail`).
- `OperationEventHandler` : nouveaux types `DEPOT_PI_SPI` (crédit compte) / `RETRAIT_PI_SPI` (débit compte, `skipComptabilisation=false`, mode `PI_SPI`).
- Schéma comptable épargne pour `PI_SPI` : manche compte client seulement (le rail publie la manche trésorerie).

1.3 Comptabilité (`sfd-comptabilite-service`)

- `plan-comptable-sfd-extension.yml` (rcs_sfd, syscohada, sycebnl) : compte de règlement PI-SPI + frais + produits ; `journaux.yml` : `PIS` (TRESORERIE).
- `TypeOperationBancaire` : `PAIEMENT_INSTANTANE_EMIS`, `PAIEMENT_INSTANTANE_RECU`.
- `CompteBancaire.typeCompte` : valeur `REGLEMENT_PI_SPI` ; `participantId`, `apiBusinessActif` (le compte bancaire du SFD chez la banque homologuée porte la config API Business côté rail, pas ici).
- `ContrepartieTresorerieRequest` : accepter `modePaiement = PI_SPI` + `endToEndId` (référence) ; `ContrepartieTresorerieServiceImpl` route `PI_SPI` → journal `PIS`.
- `RapprochementBancaireServiceImpl.importerReleve` : extraire une stratégie `ReleveParser` (`CsvReleveParser`, `OfxReleveParser`, nouveau `Camt053ReleveParser` JAXB sur `camt.053.001.x`) ; lever le rejet `rapprochement.format.nonsupporte` pour `CAMT053`.
- Préférence : `Banque` → ajouter `participantPiSpi` (bool) + `codeParticipant`.

1.4 Garde de zone UEMOA (avant tout écran ou endpoint PI-SPI)

- integration-api : `common/zone/ZoneInstitutionService.estUemoa()` (cache Caffeine/`@Cacheable` 10 min, invalidé par événement de modification d'institution/pays si disponible) + `@RequiresZone("UEMOA")` aspect ou garde explicite levant `ZoneNonEligibleException` → 403 `PISPI_HORS_ZONE`.
- Préférence : exposer la zone dans la réponse institution (`InstitutionDto.zoneMonetaire` dérivé du pays siège) pour éviter un 2e appel côté Angular.
- Angular : `NEXORA_MODULES` reçoit `zonesEligibles?: ZoneMonetaire[]` ; `paiement-instantane` = `['UEMOA']`. `ModuleRegistryService.isEnabled` = licence ∩ santé ∩ zone (zone chargée au bootstrap avec l'institution, avant `initLicensed`) ; module hors zone → ni sonde santé ni entrée menu ni route. Sélecteur `selectPispiActif` réutilisé par épargne, crédit, paie, commercial pour masquer les options PI-SPI.

- Portail client Angular + Flutter : flag `pispiActif` renvoyé par `GET /portail/config` (nouveau, ou ajouté à la réponse de login) ; écrans retrait wallet / alias / QR masqués si faux.
- Comptabilité : comptes et journal `PIS` uniquement dans `accounting-norms/uemoa/*`.
- Setup wizard : étape PI-SPI conditionnée au pays siège UEMOA.
- Tests : `ZoneInstitutionServiceTest` (UEMOA, RDC, CEMAC, zone null → refus), `module-registry.service.spec.ts` (module hors zone jamais activé, même licencié et sain), contrôleur rail renvoie 403 hors zone.

1.5 Schéma, sécurité

- Rôles : `ROLE_PISPI_READ/WRITE/ALL/ADMIN` (loader du rail), `ROLE_EPARGNE_ALIAS_*`.
- Monolithe : `MonolithSecurityConfig.PUBLIC_ENDPOINTS` + `/api/paiement-instantane/webhooks/**`.

Tests lot 1 : `AliasCompteServiceTest`, `OperationEpargneServiceRetraitIdempotenceTest`, `Camt053ReleveParserTest` (fichier exemple dans `src/test/resources/camt053-sample.xml`), `ContrepartieTresorerieServiceImplPiSpiTest`, `OutboxRelayTest`.

Lot 1 bis — Librairie `com.kodzotech:pispi-api` (3-4 semaines, en parallèle du lot 1)

Pré-requis : OpenAPI du lot 0. Repo `~/Projects/Java/commons/pispi-api` (ne jamais cloner, commons local). 1. Scaffold depuis `notification-api` (pom, Jenkinsfile, CLAUDE.md, auto-config, `*Properties`). 2. `openapi-generator-maven-plugin` sur `src/main/resources/openapi/api-business.yaml` → modèles DTO uniquement (pas de client généré, on garde `RestClient`). 3. `PiSpiTokenProvider`, `PiSpiHttpClientFactory` (mTLS par participant), `PiSpiClient` façade, exceptions typées (`PiSpiRejetException` avec code BCEAO, `PiSpiIndisponibleException`, `PiSpiHorsPerimetreException`). 4. Webhook : `PiSpiWebhookVerifier`, contrôleur optionnel `PiSpiWebhookController` (`pispi.webhook.path`, désactivable si l'appli expose le sien), dispatcher d'événements Spring. 5. Persistance optionnelle + outbox d'envoi (`@ConditionalOnProperty pispi.persistence.enabled`). 6. `TarificationPiSpi`, `PlafondPiSpi`, QR, `PiSpiSimulateur`, `PiSpiTestSupport`. 7. Tests : unitaires (token cache, signature OK/KO/rejeu, tarification, plafond), WireMock sur l'OpenAPI, test d'intégration contre la sandbox BCEAO tagué `sandbox` (exclu de Jenkins, lancé à la main avec les credentials du compte sandbox hors dépôt). 8. Publication : `./mvnw install` local puis registre Maven habituel des commons ; README d'usage hors EDIP (exemple Spring Boot minimal : payer, recevoir un webhook).

Lot 2 — Service rail `sfd-paiement-instantane-service` (5-6 semaines)

Le service EDIP dépend de `com.kodzotech:pispi-api` : il n'écrit ni client HTTP, ni OAuth2/mTLS, ni vérification de webhook, ni tarification. Il implémente `PiSpiParticipantResolver` et `PiSpiEligibilityProvider`, écoute les événements Spring de la lib, et porte tout le métier SFD.

2.1 Scaffold

- `pom.xml` depuis transfert (retirer websocket/fileupload/report si inutiles ; ajouter `spring-boot-starter-oauth2-client` non nécessaire : client_credentials fait maison via `RestClient`, + `jackson`, + `jaxb` si CAMT).

- `SfdPaiementInstantaneApplication`, `application.yml` : port 4615, contexte, datasource env, `app.rabbitmq.*` (queues `q.paiement-instantane.operations`, `q.paiement-instantane.ecritures`, routing `comptabilite.lignes.paiement-instantane`), `app.pispi.*` (ci-dessous), `app.epargne.base-url`, `app.comptabilite.base-url`, `app.client.base-url`.
- `config/{AppConfig, WebConfig, H2JpaConfig, PiSpiClientConfig, WebhookSecurityConfig, SetupPaiementInstantaneLoader}`.
- Enregistrements : `dev.sh` (`get_service`, `get_deps`=epargne,comptabilite,preference,client, `get_context_path`, `get_module_for_port`, liste ports), `sfd-erp-monolith` (`pom.xml` dépendance, `install-services.sh`, `MonolithWebConfig` préfixe `/api/paiement-instantane` ↔ package `com.synapsesit.sfd.paiementinstantane`, `MonolithModuleHealthController.MODULE_CONTEXTS`, `MonolithModuleSetupConfig` import du loader, `MonolithRabbitListenersConfig`, `application.yml` base-url), `Jenkinsfile` (`javaMicroservicePipeline`, appName `sfd-paiement-instantane`, ports 4636/4637, healthCheck), `jenkins-blueprints` si liste centrale.

2.2 Domaine

Package `com.synapsesit.sfd.paiementinstantane`. - `entity/ParticipantConfig` : `code`, `libelle`, `baseUrl`, `tokenUrl`, `clientId`, `clientSecretChiffre` (SecretCipher), `apiKeyChiffree`, `certificatKeystorePath/Password`, `webhookSecretChiffre`, `compteBancaireId` (compte de règlement/courant SFD en compta), `codeAgence` (null = institution), `actif`, `environnement` (SANDBOX|PROD). - `entity/OrdrePaiement` : `endToEndId` (UUID, unique), `clientRequestId` (unique, idempotence appelant), `sens` (SORTANT|ENTRANT), `typeFlux` (RETRAIT_EPARGNE|DEPOT_EPARGNE|PAIE|FOURNISSEUR|TRESORERIE), `montant`, `devise=XOF`, `aliasBeneficiaire`/`numeroCompteBeneficiaire`, `nomBeneficiaireVerifie`, `numeroCompteEpargne` (nullable pour institutionnel), `codeClient`, `codeAgence`, `participantCode`, `referenceParticipant`, `statut` (INITIE→ENVOYE→ACCEPTE|REJETE→IRREVOCABLE|RETOURNE|ANNULE), `motifRejet`, `fraisMontant`, `correlationId`, `moduleSource`, `payloadRequete/Reponse`, dates, `comptabilisee`. - `entity/PaiementGroupe` + `LignePaiementGroupe` (paie, fournisseurs) : `referenceGroupe`, `statut`, lignes → `OrdrePaiement`. - `entity/DemandePaiement` (request-to-pay) : `idDemande`, `aliasPayeur`, `montant`, `echeanceRef` (dossier/échéance crédit ou cotisation), `statut` (EN_ATTENTE|ACCEPTEE|REFUSEE|EXPIREE|PAYEE), `ordrePaiementId` lié à réception. - `entity/EvenementWebhook` : `idEvenement` (unique, dédup), `type`, `payload`, `signatureOk`, `traite`, `erreur`. - `entity/GrilleTarifaire` : seuil gratuit journalier (8 000), taux au-delà (0-0,8 %), frais marchand, par participant — paramétrable, jamais codé en dur. `CumulJournalier {codeClient, date, participant, montantCumule}`. - Outbox + `SagaOperation` + `OperationEventLog` (copie épargne/caisse). `EvenementWebhook` et la dédup webhook viennent de la lib (`PiSpiWebhookEvent`) ; `GrilleTarifaire` alimente `TarificationPiSpi` de la lib.

2.3 Branchement de la lib `pispi-api` (le contenu ci-dessous est fourni par la lib ; EDIP n'en garde que l'adaptateur `RailPaiementPort` → `PiSpiClient`)

- `PiSpiTokenProvider` : client_credentials sur `tokenUrl`, cache jusqu'à `expires_in - 60s`, scopes par opération.
- `PiSpiHttpClient` : `RestClient` mTLS (`SSLContext` depuis keystore PKCS12 par participant), en-têtes `Authorization: Bearer`, `X-API-Key` ; retry sur 5xx/timeout idempotent uniquement via `endToEndId`.
- `PiSpiApiBusinessAdapter` (interface `RailPaiementPort` : `initierPaiement`, `initierPaiementGroupe`, `getStatut`, `verifierAlias`, `creerAlias`, `supprimerAlias`, `creerDemandePaiement`, `retournerFonds`, `demanderAnnulation`, `enregistrerWebhook`) — mapping exact fixé par l'OpenAPI du lot 0. Un `SimulateurRailAdapter` pour tests/démo (profil `demo`/`test`).

2.4 Services

- `OrdrePaiementService.initier(OrdrePaiementRequest)` : idempotence `clientRequestId` ; contrôles (participant actif, plafond 5 M/transaction, alias vérifié + nom affiché, cumul journalier → frais) ; persist `INITIE` ; outbox `ENVOYER_ORDRE` ; relay appelle l'adapter ; MAJ statut ; publication :
- `OperationEvent` `RETRAIT_PI_SPI` / `DEPOT_PI_SPI` vers épargne (flux client) ou rien (flux institutionnel, l'appelant a déjà publié sa manche).
- Manche trésorerie via `LignesComptablesPublisherService` (compte de règlement / banque ↔ 471 attente si pas encore irrévocable ? → **non** : publier seulement à `IRREVOCABLE`, dual-line reste simple).
- `ReceptionService` (webhook `paiement.recu`) : résout alias → `AliasCompte` (épargne `GET /comptes/alias/{valeur}` ou cache local), crée `OrdrePaiement ENTRANT IRREVOCABLE`, publie `DEPOT_PI_SPI` + manche trésorerie ; si alias inconnu → `retournerFonds` + alerte.
- `WebhookService.receive(participantCode, headers, body)` : HMAC selon spec, fenêtre temps, dédup `EvenementWebhook.idEvenement`, dispatch par type (`paiement.statut`, `paiement.recu`, `demande_paiement.reponse`, `retour.recu`).
- `DemandePaiementService` : création RTP, expiration, réponse → lien avec réception.
- `ReconciliationService` : relevé participant (API `compte_transaction.read` ou CAMT.053 via compta) ↔ `OrdrePaiement` ; écarts → `EcritureEnAttenteContrepartie` compta ; rapport.
- `StatutPollingJob` : ordres `ENVOYE` > N min sans webhook → `getStatut`.
- `TarificationService.calculerFrais(codeClient, participant, montant, date)`.

2.5 API REST (rôles `ROLE_PISPI_*`)

- `/participants` CRUD (secrets write-only), `/participants/{code}/tester-connexion`.
- `/ordres` (POST initier, GET liste filtrée, GET `/{endToEndId}`, POST `/{id}/retourner`, POST `/{id}/annuler`), `/ordres/groupes` (POST, GET statut).
- `/demandes-paiement` (POST, GET).
- `/alias/verifier?valeur=` (proxy vers participant, renvoie nom + pays).
- `/webhooks/{participantCode}` (public, signé).
- `/reconciliation/{participantCode}?date=` ; `/rapports/journal-pispi`.
- `/integration/ordres` (appel service-à-service, JWT service) utilisé par épargne/paie/commercial/portail via `PaiementInstantaneRestClient`.

2.6 Consommateurs RabbitMQ

- `q.paiement-instantane.operations` : écoute `operation.retrait_pi_spi_demande` (émis par épargne quand un retrait PI_SPI est validé) → `OrdrePaiementService.initier`. Alternative plus simple retenue : épargne appelle le rail en REST synchrone (`/integration/ordres`) puis le rail publie `RETRAIT_PI_SPI` vers épargne à l'irrévocabilité. **Choix : REST pour initier, événements pour les mouvements de compte.**
- `q.paiement-instantane.ecritures` : `EcritureValideeEvent` → `comptabilisee=true` (pattern `EcritureValideeConsumer`).

2.7 Angular back-office `features/paiement-instantane`

- Vues : `participant-liste/detail` (config, test connexion), `ordre-liste/detail` (filtres statut/flux/agence, timeline statuts, payload), `paiement-groupe-liste/detail`, `demande-paiement-liste`, `reconciliation` (écarts, actions), `tableau-de-bord` (volumes, taux rejet, SLA).
- Stores NgRx `participant`, `ordre`, `paiementGroupe`, `demandePaiement`, `reconciliation`.

- Enregistrements listés dans *Conventions* ; menu `MENUITEMS.PAIEMENT_INSTANTANE.*` ; i18n fr/en/ln ; `service-map.ts` `paiementInstantane: { port: 4615, apiPrefix: '/api/paiement-instantane/', deps: ['epargne','comptabilite'] }`.
- e2e `e2e/functional/paiement-instantane/{smoke,participant-crud,ordre-flow}.spec.ts` ; script `npm run e2e:paiement-instantane` (`playwright.config.ts` projet dédié) ; `dev.sh` module.

Tests lot 2 : `PiSpiTokenProviderTest` (cache/expiry, WireMock), `PiSpiApiBusinessAdapterTest` (WireMock sur OpenAPI), `WebhookServiceTest` (signature OK/KO, rejeu), `OrdrePaiementServiceTest` (idempotence, plafond, frais, transitions), `ReceptionServiceTest` (alias inconnu → retour), `OrdrePaiementControllerTest`, `OrdrePaiementRepositoryTest`, `SfdPaiementInstantaneApplicationTests`, `PostgresSchemaTest`/`MssqlSchemaTest`.

Lot 3 — Flux métier et canaux clients (4-5 semaines)

3.1 Épargne : retrait/dépôt PI-SPI

- `OperationEpargneService.effectuerRetraitPiSpi(RetraitPiSpiRequestDto{numeroCompte, montant, alias|numeroCompteBeneficiaire, libelle, clientRequestId})` : contrôle solde + plafonds client, **réserve** le montant (opération `EN_ATTENTE`, mode `PI_SPI`), appelle `PaiementInstantaneRestClient.initierPaiement` ; à `RETRAIT_PI_SPI` irrévocable (événement) → `VALIDEE` + manche compte ; à `REJETE` → annulation réservation. Endpoints `POST /operations/retrait-pi-spi` (back-office) et `POST /portail/comptes/retraits-pi-spi` (client, OTP obligatoire).
- Dépôt entrant : `OperationEventHandler` `DEPOT_PI_SPI` → `effectuerDepot(mode PI_SPI, clientRequestId=endToEndId)`.

3.2 Crédit

- `DossierCreditServiceImpl.validerConditionsDecaissement` : autoriser `VIREMENT_EXTERNE` et `MOBILE_MONEY` **uniquement comme chaînage** : décaissement inchangé sur compte épargne (`VIREMENT_COMPTE`), puis `DecaissementRemiseService.appliquer` (aujourd'hui no-op) déclenche `EpargneRestClient.effectuerRetraitPiSpi(numeroCompteEpargne, montant, alias/IBAN du` DecaissementRequest`)`. Ajouter `aliasBeneficiaire`, `iban`, `bic` à `DecaissementRequest` (collectés par l'UI `decaissement-modal`, aujourd'hui perdus). Échec du retrait chaîné = décaissement reste valide, alerte, retrait rejouable.
- Remboursement : inchangé (`REMBOURSEMENT_CREDIT` débite l'épargne). Ajout `PrelevementAutomatiqueServiceImpl.emettreDemandesPaiement()` : pour échéances `PREVUE` J-3 dont le compte épargne est insuffisant, créer une `DemandePaiement` PI-SPI (alias client) ; réception → dépôt épargne → prélèvement existant.
- Supprimer la logique `resolveCompteBanque 521` de `DecaissementRemiseService` (compte de repli interdit).

3.3 Paie

- `PaiementSalaireService` : domiciliation `BANQUE` / `MOBILE_MONEY` → au lieu de la simple contrepartie, construire un `PaiementGroupe` (lignes = employés avec `iban`/`compteMobileMoney`/alias) via `PaiementInstantaneRestClient.initierPaiementGroupe` ; manche Paie 421 + contrepartie trésorerie

publiées à l'irrévocabilité (consommateur `EcritureValideeEvent`/webhook statut groupe). `ExportVirementService` conservé comme repli.
- `PayerBulletinRequest` : `canal` (`PI_SPI|FICHIER`).
- RH `Employe` : champ `aliasPiSpi` optionnel.

3.4 Commercial

- `TiersCommercial` : `iban`, `banqueId`, `aliasPiSpi` (manquants) + écran tiers Angular.
- `ReglementTresorerieService.regler` : mode `VIREMENT/MOBILE_MONEY` en décaissement fournisseur → ordre PI-SPI institutionnel ; encaissement client → via dépôt épargne si le tiers est client, sinon réception sur compte bancaire SFD (manche tiers à la réception).

3.5 Portail client (Angular + service)

- `PortailRetraitMobileService` : remplacer le stub par `OtpService.verify(userId, otp, VIREMENT)` + `EpargneRestClient.effectuerRetraitPiSpi` ; `RetraitMobileMoneyRequest` : `operateur` → `alias` (téléphone) ; réponse = `endToEndId` + statut réel ; `GET /retraits-mobile/{ref}` suivi.
- `PortailVirementService` : virement externe par alias (`POST /virements/externe`), vérification nom bénéficiaire (`/alias/verifier`), frais depuis `TarificationService`.
- `GET /comptes/{n}/rib` fonctionnel + `GET /comptes/{n}/qr` (QR PI-SPI de réception, SDK QR du portail BCEAO).
- Angular `sfd-portail-client-angular/features/portail/view/{retrait-mobile, virement-externe, rib-qr}` + stores.
- Flutter `sfd-mobile/sfd_portail_client/lib/features/mobile_money` : modèle `alias`, nouveau statut, écran suivi ; corriger `virements` → `virements/executer` ; **ignorer la copie divergente `sfd_portail_client/` racine** (à supprimer ou aligner, hors plan).

3.6 Transfert

- `sfd-transfert-service` : `TRF_NAT` exécuté via rail (ordre institutionnel depuis compte bancaire SFD, bénéficiaire alias) ; `TRF_UEM` désactivé jusqu'au 01/06/2027 (flag).

Tests lot 3 : `OperationEpargneServiceRetraitPiSpiTest`, `DossierCreditServiceImplDecaissementChaineTest`, `PaiementSalaireServicePiSpiTest`, `PortailRetraitMobileServiceTest` (OTP KO/OK), e2e `epargne-retrait-pispi.spec.ts`, `credit-decaissement-externe.spec.ts`, `paie-virement-groupe.spec.ts`, portail e2e retrait + RIB.

Ordre d'exécution et dépendances

1. Lot 0 (bloquant pour la lib : OpenAPI). 1 bis. Lot 1 bis `pispi-api` en parallèle du lot 1 (repos disjoints) ; publiée avant le lot 2.
2. Lot 1.1 integration-api → publier `1.0.2-SNAPSHOT` → 1.2 épargne, 1.3 compta en parallèle → 1.4.
3. Lot 2.1-2.6 backend rail (adapter contre WireMock + sandbox BCEAO) → 2.7 Angular.
4. Lot 3 : 3.1 épargne, puis 3.5 portail (dépend 3.1), 3.2 crédit, 3.3 paie, 3.4 commercial, 3.6 transfert indépendants.
5. Push par dépendance (mémoire `push-order-dependency-rule`) : pispi-api (commons) → integration-api → épargne → compta → rail → crédit/paie/commercial/portail → monolithe → Angular.

Vérification (par lot, règles CLAUDE.md)

- Backend : `./mvnw test` 0 échec par service touché ; `./mvnw clean package -DskipTests -o -q` ; démarrage sans profil + `/actuator/health` UP (rail sur 4615).
- Contrats partagés : grep des appelants de `ModeOperation`, `ModeDecaissement`, `TypeOperationBancaire`, `ContrepartieTresorerieRequest`, `OperationEvent` après chaque ajout d'enum ; test de régression sur `validerConditionsDecaissement` (ESPECES/CHEQUE toujours refusés).
- Sandbox BCEAO : scénario complet contre le participant simulateur (alias → réception → retrait → webhook → réconciliation) documenté dans `sfd-docs/refs/pispi/TEST_SANDBOX.md`.
- Angular : `npm run build` 0 erreur TS ; `./dev.sh start paiement-instantane` + `npm run e2e:paiement-instantane` 0 skipped ; `npm run e2e:epargne`, `e2e:credit`, `e2e:paie` pour les flux chaînés.
- Hors zone (obligatoire) : base démo RDC (wizard démo IFOD RDC) → aucun menu PI-SPI, route `/paiement-instantane` refusée, aucune option PI-SPI dans épargne/crédit/paie/portail, appel direct `POST /api/paiement-instantane/ordres` → 403 `PISPI_HORS_ZONE` ; e2e `paiement-instantane-hors-zone.spec.ts`.
- Navigateur interne (obligatoire) sur base démo jetable (`edip-env.sh up pispi --baseline <rN>`) : retrait portail → ordre visible back-office → statut irrévocable (simulateur) → mouvement compte épargne → pièce comptable équilibrée (règlement PI-SPI / compte client) → réconciliation sans écart.
- Comptabilité : chaque flux produit une pièce équilibrée uniquement à l'irrévocabilité ; aucun compte de repli ; deux positions trésorerie distinctes (règlement clients vs compte courant institutionnel) rapprochées séparément.

Risques suivis

- Spec API Business non encore lue : l'adapter est isolé derrière `RailPaiementPort`, seul `PiSpiApiBusinessAdapter` change.
- Irrévocabilité : outbox + idempotence `endToEndId`/`clientRequestId` avant tout appel sortant ; jamais de double envoi en retry sans `endToEndId`.
- Secrets : chiffrés `SecretCipher`, jamais en dépôt ; keystore mTLS monté par volume (Jenkins/VPS).
- Monolithe : `WebhookSecurityConfig` hors package `config` (sinon exclu par le scan) + `PUBLIC_ENDPOINTS`.
- Grille tarifaire BCEAO révisée le 02/10/2026 : paramétrage, pas de constante.